Variables and Assignment in Python

Overview:

- **Variables** are containers for storing data values. In Python, they act as references to information stored in memory.
- The **assignment operator** = is used to assign values to variables.
  - Example: x = 1 assigns the value 1 to the variable x.
- Variables in programming are unlike mathematical variables; they do not represent a truth statement but rather a storage location for data.

Key Concepts:

- Variables are named using alphanumeric characters and underscores (_).
- The first character of a variable name must be a letter or underscore.
- Variable names are **case-sensitive** (e.g., x and X are different).
- Avoid using names of built-in functions as variable names (e.g., help = 2).

Example:

```python
x = 1
y = 2
y = y * 3  # y now behaves as 2, and y * 3 equals 6
```

Assignment vs. Mathematical Equality:

- In Python, x = 2 assigns the value 2 to x. It's different from the mathematical statement x = 2, which is a truth assertion.
- Python assignments always go left: the right-hand value is stored in the left-hand variable.

Important Operations:

- Example: x = x + 1 in programming:
  - It updates x by adding 1 to its current value.
  - If x was 1 before, it becomes 2 after the operation.

Managing Variables:

- Use %whos to list all variables in memory.
- Clear a specific variable using del variable_name.
- Clear all variables with %reset.

Example:

```python
x = 1
x = x + 1  # Now, x equals 2
del x  # x is removed from memory
```

Best Practices in Variable Naming and Usage

Naming Conventions:

- Variables should have meaningful names that reflect their purpose.
  - Example: Use distance for the distance between two points instead of x.
- Avoid spaces in variable names; use underscores or camelCase for readability.

Caution:

- Reassigning a variable does not retain any memory of how it was assigned.
  - Changing x after y = x + 1 does not affect y.
- Be mindful of overwriting variables or functions (e.g., reassigning help).

Example:

```python
distance = 5
numRabbits = 12
```

TIP: Always assign results to variables to keep track of operations and avoid confusion.

Strings in Python

What is a String?

- A string is a sequence of characters enclosed within single or double quotation marks.
- Examples: "Hello World", 'Python', "12345".

Key Characteristics:

- **Type Verification**: Strings can be verified using the type() function.
  - Example:

```python
w = "Hello World"
type(w)  # Output: <class 'str'>
```

- **Empty Strings**: You can create an empty string and verify it.
  - Example:

```python
empty_str = ""
type(empty_str)  # Output: <class 'str'>
```

String Length and Indexing:

- **Length**: Use len() function to find the number of characters in a string.
  - Example:

```python
len(w)  # Output: 11
```

- **Indexing**: Strings are indexed starting from 0. You can access any character using its index.
  - Example:

```python
w[6]  # Output: 'W'
```

Advanced String Operations in Python

String Slicing:

- **Basic Slicing**: Extract a substring using the syntax [start:end].
  - Example:

```python
w[6:11]  # Output: 'World'
```

- **Negative Indexing**: Use negative indices to slice from the end of the string.
  - Example:

```python
w[-5:]  # Output: 'World'
```

String Manipulation Methods:

- **Concatenation**: Combine two strings using the + operator.
  - Example:

```python
str_a = "Hello"
str_b = "World"
print(str_a + " " + str_b)  # Output: 'Hello World'
```

- **Upper Case Conversion**: Convert all characters in a string to upper case.
  - Example:

```python
w.upper()  # Output: 'HELLO WORLD'
```

- **String Replacement**: Replace a substring within a string.
  - Example:

```python
w.replace("World", "Python")  # Output: 'Hello Python'
```

Formatted Strings (f-Strings):

- **Basic Usage**: Insert variables into strings easily.
  - Example:

```python
name = "Python"
print(f"I love {name}!")  # Output: 'I love Python!'
```

- **Expression Evaluation**: Directly evaluate expressions inside strings.
  - Example:

```python
print(f"{3 * 4}")  # Output: '12'
```

Lists in Python

- **Definition and Characteristics**
  - Python lists are versatile data structures that can store a sequence of items.
  - Lists are defined using square brackets [ ], with elements separated by commas.
  - A list can contain any type of data, including numbers, strings, and even other lists (nested lists).
- **Examples**
  - A simple list: list_1 = [1, 2, 3]
  - A list with mixed data types: list_3 = [1, 2, 3, 'Apple', 'orange']
  - Nested lists: list_4 = [[1, 2, 3], ['Hello', 'World']]
- **Accessing Elements**
  - Elements are accessed using indices, similar to strings.
  - Example: list_3[2] retrieves the 3rd element of list_3, which is 3.

Operations on Lists

- **Basic Operations**
  - **Length**: Use len(list) to get the number of elements in a list.
  - **Concatenation**: Combine lists using the + operator.
  - **Appending**: Add elements to a list using append().
  - **Inserting**: Insert elements at a specific position using insert().
  - **Deleting**: Remove elements by index using the del statement.

- **Examples**
  - Adding lists: list_1 + list_2 results in [1, 2, 3, 'Hello', 'World'].
  - Appending: list_1.append(4) changes list_1 to [1, 2, 3, 4].
  - Inserting: list_1.insert(2, 'center') changes list_1 to [1, 2, 'center', 3, 4].
  - Deleting: del list_1[2] removes the element at index 2, resulting in [1, 2, 3, 4].

Advanced List Operations

- **Checking Membership**
  - Use the in operator to check if an element exists in a list.
  - Example: 5 in list_5 returns True if 5 is in list_5.
- **Converting Sequences to Lists**
  - The list() function can convert other sequence types (e.g., strings) into lists.
  - Example: list('Hello World') converts the string into a list of characters: ['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd'].
- **Usage in Data Handling**
  - Lists are frequently used in Python for handling and manipulating data, making them a fundamental tool in programming.

Tuples in Python

- **Definition**: A *tuple* is a sequence data structure in Python, typically defined using parentheses ( ). Elements within a tuple are separated by commas.
- **Key Characteristics**:
  - **Immutability**: Once a tuple is created, its elements cannot be modified.
  - **Heterogeneity**: Tuples can store a heterogeneous sequence of elements, meaning they can contain elements of different data types.
- **Comparison with Lists**:
  - Tuples are immutable, while lists are mutable.
  - Tuples often store a fixed sequence of diverse elements, whereas lists usually contain a sequence of homogeneous elements.

Example:

```python
tuple_1 = (1, 2, 3, 2)
print(tuple_1[1:3])  # Output: (2, 3)
```

Operations and Uses of Tuples

- **Common Operations**:
  - **Indexing and Slicing**: Similar to lists, tuples support indexing and slicing.
    - Example: tuple_1[1:4] gives (2, 3, 2)
  - **Length**: len(tuple_1) returns the number of elements.
  - **Counting Elements**: tuple_1.count(2) returns the count of a specific element.

- **Use Cases**:
  - **Data Integrity**: Tuples are ideal when you need a data structure that should not be modified.
  - **Unpacking**: Tuples can be easily unpacked into variables.
    - Example: a, b, c = tuple_1
  - **Heterogeneous Data**: Used to store pairs of related but different data types, like (fruit, quantity) pairs.

Example:

```python
fruits = ('apple', 3)
print(f"Fruit: {fruits[0]}, Quantity: {fruits[1]}")  # Output: Fruit: apple, Quantity: 3
```

Practical Examples and Tuple Constraints

- **Immutability in Practice**:
  - Attempting to modify a tuple element results in a TypeError.
  - Example:

```python
tuple_1 = (1, 2, 3)
tuple_1[2] = 4  # Raises TypeError: 'tuple' object does not support item assignment
```

- **Packing and Unpacking**:
  - Tuples support *packing* (combining multiple values into a tuple) and *unpacking* (assigning tuple elements to variables).

- **Real-World Scenario**:
  - Example: Storing coordinates (x, y) as a tuple for geometric operations.
  - Example: Packing items into a tuple point = (10, 20) and then unpacking x, y = point.

Conclusion:

- Tuples are a crucial data structure in Python, offering immutability and a convenient way to handle related but varied data types.

Sets in Python

- **Definition**:
  - Sets in Python are an unordered collection of unique elements, defined using curly braces {}.
  - They do not allow duplicate elements, making them ideal for operations requiring uniqueness.
- **Creation**:
  - Example: set_1 = set([1, 2, 2, 3, 2, 1, 2]) results in {1, 2, 3}.
  - A set can be created from a list, tuple, or string by passing it to the set() function.
- **Key Properties**:
  - Unordered: No guaranteed order of elements.
  - Immutable: Elements inside the set cannot be changed, but the set itself is mutable (i.e., elements can be added or removed).

Basic Operations on Sets

- **Finding Unique Elements**:
  - Example: set([1, 2, 2, 3, 2, 1, 2]) → {1, 2, 3}
  - Example: set('Banana') → {'B', 'a', 'n'}
- **Mathematical Operations**:
  - **Union**: Combines elements from both sets without duplicates.
    - Example: set_1.union(set_2) where set_1 = {1, 2, 3} and set_2 = {2, 4, 5, 6} results in {1, 2, 3, 4, 5, 6}.
  - **Intersection**: Returns elements that are present in both sets.
    - Example: set_1.intersection(set_2) results in {2}.
  - **Difference**: Returns elements that are in one set but not the other.
    - Example: set_1.difference(set_2) results in {1, 3}.
  - **Symmetric Difference**: Elements that are in either set, but not in both.
    - Example: set_1.symmetric_difference(set_2) results in {1, 3, 4, 5, 6}.

Advanced Set Operations and Applications

- **Subset and Superset**:
  - **Subset**: Check if a set is a subset of another.
    - Example: set_1.issubset({1, 2, 3, 4, 5}) returns True because all elements of set_1 are in the larger set.
  - **Superset**: Check if a set contains another set.
    - Example: {1, 2, 3, 4, 5}.issuperset(set_1) returns True.
- **Applications**:
  - Sets are used to filter unique items, perform fast membership testing, and implement common mathematical operations in Python programs.
  - Examples include removing duplicates from a list, performing data analysis to find common items across multiple datasets, and working with collections where order is not important.

Dictionaries in Python

- **Definition**:
  - A **dictionary** in Python is a collection of key-value pairs where each key is unique.
  - Unlike sequences (lists or tuples), dictionaries are **unordered** and indexed by keys rather than by a numerical index.
- **Key Features**:
  - **Keys**: Can be strings, numbers, or tuples but cannot be lists.
  - **Values**: Can be of any data type.
  - **Syntax**: Defined using {}, with key-value pairs separated by colons.

```python
dict_1 = {'apple': 3, 'orange': 4, 'pear': 2}
```

Accessing Elements:

- Access a value by referring to its key:

```python
dict_1['apple']  # Output: 3
```

Working with Dictionaries

- **Common Operations**:
  - **Getting all keys**: dict_1.keys()
  - **Getting all values**: dict_1.values()
  - **Getting dictionary length**: len(dict_1)

- **Creating and Modifying Dictionaries**:
  - **Empty Dictionary**: Can be created and filled later.

```python
school_dict = {}
school_dict['UC Berkeley'] = 'USA'
```

  - **Converting Tuples to Dictionary**:

```python
dict([("UC Berkeley", "USA"), ('Oxford', 'UK')])
```

- **Checking Membership**:
  - Check if a key exists using the in keyword:

```python
"UC Berkeley" in school_dict  # Output: True
```

  - To check if a key does not exist, use not in:

```python
"Harvard" not in school_dict  # Output: True
```

Practical Examples and Use Cases

- **Practical Usage**:
  - Dictionaries are highly useful for **mapping** data where a unique identifier (key) maps to a specific value.
  - Examples include storing **user data** by ID, **configuration settings**, and **look-up tables**.
- **Examples**:
  - **Inventory System**:

```python
inventory = {'apple': 50, 'banana': 30, 'orange': 20}
```

  - **Gradebook**:

```python
grades = {'Alice': 'A', 'Bob': 'B+', 'Charlie': 'A-'}
```

Advantages:

- **Efficient Lookups**: Access elements in constant time.
- **Flexible Data Handling**: Can store complex data structures as values.

Introduction to Numpy Arrays

Overview:

•Numpy is a fundamental Python module for numerical computing, widely used in scientific computing.

•Key features of Numpy:

- **N-dimensional Array Object:** Efficiently handles large datasets with a powerful array object.
- **Broadcasting:** Facilitates operations on arrays of different shapes.
- **Integration with C/C++ and Fortran:** Enhances performance by integrating with lower-level languages.
- **Linear Algebra, Fourier Transform, and Random Number Capabilities:** Essential tools for scientific computing.

Creating Numpy Arrays:

•Importing Numpy: import numpy as np (community convention uses “np” as the alias).

•Converting a list to an array: np.array([1, 4, 3]) results in array([1, 4, 3]).

•2D Arrays: Use nested lists, e.g., np.array([[1, 4, 3], [9, 2, 7]]).

Array Attributes:

•shape: Returns a tuple of the array dimensions. E.g., (2, 3) for a 2x3 matrix.

•size: Returns the total number of elements in the array.

Numpy Array Operations

Generating Arrays:

- **Using np.arange:** Creates arrays with a defined start, end, and step value.
  - Example: np.arange(1, 2000, 1) generates [1, 2, 3, ..., 1999].
- **Using np.linspace:** Creates arrays with a specified number of evenly spaced elements.
  - Example: np.linspace(3, 9, 10) generates [3, 3.67, 4.33, ..., 9].

Array Indexing:

- 1D Array: Access elements using indices, e.g., x[1] for the second element.
- 2D Array: Access elements using row and column indices, e.g., y[0, 1] for the element in the first row, second column.

Common Array Operations:

- **Arithmetic Operations:** Perform element-wise operations between arrays and scalars.
  - Example: b + 2 adds 2 to each element of array b.
- **Matrix Multiplication:** Element-by-element multiplication using *.
  - Example: b * d multiplies corresponding elements of arrays b and d.